Intent, Custody, Trajectory, and Proof:
Toward Accountable Execution in Agentic Software Engineering

Alex H. Raber

Decapod Labs

alexhraber@gmail.com

Technical Report — July 2026

Abstract

Large language model (LLM) agents are transitioning from conversational programming assistants to autonomous executors capable of editing codebases, running build tools, executing tests, and managing multi-file software deployments. Despite this shift, current agent workflows still rely on standard prompt transcripts and code diffs as the primary logs of execution. As agent runs become concurrent, long-running, and tool-intensive, this transcript-only paradigm exposes repositories to significant failure modes: lost intent, authority leakage, concurrent state corruption, and unverifiable completion claims.

This paper frames agentic software engineering as a distributed execution problem. We introduce a framework built around four primitives to achieve accountable execution: *Intent* (durable task objectives and constraints), *Custody* (task ownership and isolated workspace boundaries), *Trajectory* (event-backed records of governance activity), and *Proof* (machine-checkable validation and provenance evidence). We present Decapod, a daemonless, local-first governance kernel, as a reference implementation of this model. The accompanying artifact defines an empirical evaluation protocol over 35 multi-step software-engineering tasks, but the quantitative results in this report are not yet empirical: checked-in summaries are explicitly synthetic and must not be interpreted as measurements. We state the resulting hypotheses and the controls needed to test them without claiming that governance improves model capability or guarantees security.

1 Introduction

The integration of Large Language Models (LLMs) into software engineering has progressed rapidly through three distinct phases: suggestion (e.g., inline autocompletion), instruction (e.g., conversational chat interfaces), and delegation (e.g., autonomous agents). In this current phase of delegated execution, an agent is not merely a generator of code snippets; it is an active developer. Equipped with a tool belt of shell executors, file readers, linters, compilers, and test suites, the agent operates directly on target repositories, running iterative loops to edit files, compile binaries, parse build logs, and debug errors.

However, as agentic workflows evolve from simple scripts to long-running, concurrent, multi-agent systems, the mechanisms used to control and record their execution remain dangerously primitive. Most contemporary agent frameworks (e.g., AutoGPT,

Devin-like implementations, and simple chat wrappers) treat the natural language prompt, the conversational chat transcript, and the final git diff as the sufficient record of work.

We argue that this transcript-only paradigm is fundamentally inadequate for serious software engineering environments. Chat transcripts are verbose, non-deterministic, and prone to context window drift. More critically, they lack clear boundaries of authority, fail to capture environment validation results in a machine-checkable format, and provide no guarantees that code changes satisfy the intended constraints. Once an agent is granted write access to a repository and the authority to run arbitrary CLI tools, software engineering transforms from a language-modeling problem into a **distributed execution and security problem**.

When viewed through a systems-oriented lens, agentic software engineering shares core challenges with classic distributed systems:

- **Authority & Isolation**: How do we ensure that an agent only mutates the directories it is authorized to touch, without leaking credentials or accessing sensitive environment variables?
- **Concurrency**: How do we prevent multiple concurrent agents or human developers from corrupting the workspace, modifying overlapping code paths, or introducing uncoordinated merge conflicts?
- **Failure & Recovery**: If an agent runs out of tokens, hits rate limits, or crashes mid-task, how can a subsequent agent or human developer recover the execution state and continue without starting from scratch?
- **Auditability**: How can a human developer efficiently verify that an agent's changes are correct without manually reviewing thousands of lines of conversational LLM reasoning?

To address these challenges, this paper formalizes four core primitives for accountable execution:

1. **Intent**: Durable task objectives, constraints, open questions, and proof expectations represented outside the model's transient context.
2. **Custody**: Exclusive task ownership and isolated git-worktree or container boundaries within which the agent operates.
3. **Trajectory**: Event-backed records of task, broker, proof, and governance activity that can be rendered as a flight-recorder timeline.
4. **Proof**: Programmatically checkable validation, proof-run, workunit, and provenance evidence; cryptographic signing is a future extension, not an implemented claim here.

We present **Decapod**, a daemonless, local-first governance kernel, as a reference implementation of this model. We propose an empirical evaluation protocol to analyze governed execution across 35 multi-step software tasks under three conditions: prompt-only baseline, checklist-disciplined baseline, and Decapod-governed treatment. The contribution at this stage is the model, implementation mapping, and falsifiable protocol—not a measured performance result.

2 Background and Lineage

The transition from static algorithms to autonomous software engineering agents is underpinned by a century-long chain of intellectual developments in computation, information, learning, scale, and distributed systems. Understanding this lineage is essential to framing agentic software engineering not as an ad-hoc scripting trend, but as a formal systems execution domain.

2.1 Formalizing the Machine and the Message

The mathematical foundation of computing begins with Turing's formalization of computation in 1936 [12]. By introducing the Turing Machine, Alan Turing defined the limits of what can be computed algorithmically and established that a universal machine could simulate any other process given a discrete set of rules and an infinite tape.

Twelve years later, Shannon defined what flows through these machines [11]. In *A Mathematical Theory of Communication*, Claude Shannon stripped meaning from language, formalizing information as a measurable quantity of entropy and establishing the bit as the core unit of channel capacity. This mathematical framing of entropy and prediction under uncertainty directly underpins the loss functions and token-generation architectures used in modern language models.

2.2 The Connectionist Evolution and Distributed Ordering

Early attempts to make machines learn from their environments began with Rosenblatt's Perceptron in 1958 [9], which showed that machines could adjust numerical weights to classify input patterns. However, Minsky and Papert's critique in 1969 [7] exposed the geometric limitations of single-layer networks, notably their inability to learn non-linearly separable functions like XOR. This critique initiated the first AI winter, which was resolved seventeen years later by Rumelhart, Hinton, and Williams [10] who formalized backpropagation, showing that stacking layers of perceptrons and using the chain rule from calculus allowed multi-layered networks to learn complex internal representations.

Concurrently, the systems substrate of computing was expanding from single processors to distributed networks. Lamport's seminal 1978 paper [6] established that without a shared physical clock, events in a distributed system must be ordered based on causality (the "happens-before" relation) rather than wall-clock time. This conceptual breakthrough made large-scale distributed databases, cloud storage, and eventually parallel GPU training clusters possible, preventing concurrent machines from dissolving into state inconsistency.

2.3 Scale, Attention, and Emergent Intelligence

In 1998, Brin and Page demonstrated that relevance could emerge from planetary-scale graph structure through the PageRank algorithm [1]. This proved that indexing and ranking massive piles of human text and links could yield high-quality search interfaces. By 2012, the ImageNet breakthrough (AlexNet) by Krizhevsky et al. [5] proved that connectionist backpropagation, when combined with massive datasets and consumer-grade graphics processing units (GPUs), dramatically outperformed hand-engineered features in computer vision.

The modern era of LLMs was ignited by Vaswani et al. in 2017 [13] with the Transformer architecture. By discarding sequential recurrent networks in favor of parallelizable

attention mechanisms, the Transformer allowed models to compute associations between all tokens in a context window simultaneously. Finally, in 2020, Brown et al. showed in GPT-3 [2] that scaling these transformers to hundreds of billions of parameters transforms language predictors into general-purpose few-shot learners. GPT-3 proved that scale turns predictions into programmable interfaces.

2.4 The Agentic Transition and the Accountability Gap

While GPT-3 and subsequent models excel at static text completion, software engineering requires active intervention. This transition from static representation to dynamic action was foreshadowed by Mnih et al. in Deep Q-Networks (DQN) [8], where neural networks were integrated into closed-loop reinforcement learning environments—mapping perception directly to actions and environment feedback.

Today, LLM agents combine the semantic reasoning of transformers with the action loops of reinforcement learning, executing commands via shell tools over mutable code repositories. Yet, as computation has transitioned from Turing's abstract tape, through Shannon's bits, Lamport's clocks, and Transformer attention, we have reached a critical systems gap: **we have executable intent, but we lack custody and proof**. This paper bridges this gap, developing the control plane primitives needed to make autonomous agent runs safe and accountable.

3 Problem Formulation

The standard model of agentic software engineering relies on a chat-based transcript-only workflow. The agent is initialized with a system prompt and a user command, interacts with files and commands through tool wrappers, and logs its output in a conversational transcript. We formalize five distinct failure modes inherent in this architecture.

3.1 Intent Loss (State Loss Under Interruption)

When an agent's process is interrupted due to network timeouts, model rate limits, or crash events, the current state of its work is lost. In a transcript-only workflow, there is no separate, structured record of progress. Resuming the task requires either:

1. Replaying the entire transcript from the beginning, which is slow and expensive.
2. Launching a new agent in the modified workspace. Because the new agent has no access to the previous agent's reasoning, it must guess the intent from the mutated files. This often leads to the agent reverting partially finished edits or introducing redundant code.

3.2 Context Drift (Loss of Task Focus)

LLMs operate within a finite context window. As an agent runs multi-step tasks (e.g., executing commands, reading compiler stack traces, parsing log files), the context window becomes saturated with terminal stdout/stderr. As the initial prompt and instructions are pushed out of the active context, the agent suffers from context drift: it forgets original constraints, ignores boundary rules, and enters infinite loops of repeating the same failing tool commands.

3.3 Authority Leakage (Unbounded Privileges)

Transcript-only workflows do not partition credentials or system permissions. The agent runs with the privileges of the parent terminal. If the agent receives a prompt to debug a web application, but is exposed to malicious input (e.g., via a dependency or a test database file), prompt injection or model hallucination can cause the agent to execute destructive commands (e.g., `rm -rf /`) or access sensitive environmental credentials (e.g., AWS tokens) that are unrelated to the target task.

3.4 Concurrent Workspace Collision (State Inconsistency)

In active development environments, multiple agents or human engineers work concurrently on the same codebase. In a transcript-only workflow operating directly on a single shared repository directory, concurrent edits cause state corruption. If Agent A compiles the code while Agent B is modifying a dependency, the build output is non-deterministic. Furthermore, without logical branch isolation, agents overwrite each other's files, leading to silent code regressions and complex merge conflicts.

3.5 Unverifiable Completion (False Success Claims)

Agents frequently hallucinate completion. When an agent runs into a hard debugging loop, it often stops executing tools and asserts verbally in the chat transcript: "I have successfully resolved the issue." However, the repository remains in a broken or incomplete state. Without an independent, machine-checkable verification gate that must be passed before a completion claim is registered, these false-positive assertions require humans to manually pull the code, compile it, and run tests themselves to check the agent's work.

4 Formal Execution Model

To resolve the five failure modes identified in Section 3, we formalize an execution model built around Intent, Custody, Trajectory, and Proof. This model abstracts agentic software engineering as a state transition system operating over a repository space.

Let R represent the state of the repository, including the files, directory structure, git history, and runtime environment. An agent run is a sequence of state transitions $R_0 \xrightarrow{a_1} R_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} R_n$, where each action a_i is executed by the agent via tools.

4.1 Intent: The Goal Objective

We define Intent (I) as a durable task contract containing goals, constraints, unresolved questions, stop conditions, and proof expectations:

$$I = \langle G, C, V \rangle$$

where $G = \{g_1, g_2, \dots, g_k\}$ represents the goals, C represents explicit constraints, and $V = \{v_1, v_2, \dots, v_m\}$ represents proof hooks or validation expectations. In Decapod, these concerns are distributed across governed plan artifacts, todo records, project specs, and proof configuration rather than a single universal `intent.json`. The contract persists outside the model context, allowing later steps to re-resolve intent and detect unresolved questions before execution.

4.2 Custody: Environment Bounding

Custody (K) defines the execution and security sandbox for the agent. Formally, it is a tuple:

$$K = \langle W, B, A, P \rangle$$

where:

- $W \subset R$ is the isolated workspace slice (a task-scoped git worktree and, when configured, a container).
- B is the task-specific branch locked for the agent.
- A is the set of governance operations and proof capabilities available to the run.
- P represents the repository and runtime boundary enforced by the workspace configuration.

By enforcing custody boundaries, the kernel keeps the agent's repository mutations in W and prevents direct work on protected branches. This reduces cross-task workspace collisions; it is not a complete operating-system sandbox or credential-isolation proof.

4.3 Trajectory: The Governance Record

The Trajectory (T) is an event-backed sequence of governance records:

$$T = [t_1, t_2, \ldots, t_n]$$

where a record may include timestamp, operation, actor, session, correlation identifier, status, and structured details. Decapod journals todo, broker, proof, and related governance events and can render them through its flight recorder. These records improve reconstruction and review, but they are not asserted here to be a complete capture of every model token, shell byte, file diff, or tamper-proof execution event.

4.4 Proof: Completion Evidence

A Proof (P_f) is the set of machine-checkable evidence attached to a governed task:

$$P_f = \langle I, R_n, \{\text{Pass}(v_j)\}, E \rangle$$

where E contains relevant proof events, workunit results, validation output, and provenance references. In Decapod, validation and proof gates can block promotion or completion paths when required evidence is absent. A proof record establishes what the configured gates observed; it does not establish semantic correctness beyond those gates, and this paper makes no claim that the current implementation signs a universal certificate.

5 Decapod: A Reference Governance Kernel

Decapod is a daemonless, local-first governance kernel for AI coding agents. It is invoked by agents at governance boundaries; it does not replace the model's tool loop or claim to make the underlying model more capable. The implementation mapping below is based on the current Decapod source tree and generated project specifications.

5.1 System Architecture

The CLI and JSON-RPC surfaces route requests into core subsystems that persist state under `.decapod/`. The principal surfaces relevant to this paper are:

1. **Intent and coordination**: governed plans capture intent, unknowns, human questions, stop conditions, constraints, todo identifiers, and proof hooks. The todo subsystem records task ownership, claims, dependencies, and events.
2. **Custody**: workspace management creates todo-scoped git worktrees and, when configured, container workspaces. Protected branches and workspace checks prevent an agent from treating the human root checkout as its execution surface.
3. **Trajectory**: todo, broker, proof, and related event journals are rendered by the flight recorder as a timeline or transcript. This is a governance record, not a complete capture of all model or shell activity.
4. **Proof and validation**: configurable proof commands produce structured results and proof events; `decapod validate`, workunit gates, provenance artifacts, and verification checks supply promotion evidence.

The generated specs describe the same corridor as intent $\rightarrow$ claim $\rightarrow$ workspace $\rightarrow$ proofs $\rightarrow$ publish. The repository is local-first and cloud-disabled in the evaluated configuration; external systems are optional publication or coordination surfaces rather than the local source of truth.

5.2 Intent and Governed Workflow

Decapod does not currently require a universal `.decapod/intent.json` manifest or file-marker completion protocol. Instead, the workflow is distributed across explicit interfaces such as:

```
decapod session acquire
decapod todo add "task description"
decapod todo claim --id <task-id>
decapod infer orientation --intent "..." --task-id <task-id>
decapod workspace ensure --container
decapod validate
decapod todo done --id <task-id> --validated
```

Governed plans add a stronger pre-execution contract: execution is blocked when the plan is not approved, when intent or unknowns remain unresolved, or when no todo is selected. These mechanisms operationalize durable intent without implying that every natural-language requirement is machine-verifiable.

5.3 Proof and Evidence Boundaries

Proof definitions contain commands, arguments, descriptions, and required status. A proof run records exit status, duration, pass/fail state, bounded output, run identifiers, actor, and related requirements in the proof event journal. Workunit and workspace-publish gates can require proof results and validation evidence before promotion.

The current implementation therefore supports auditable, machine-checkable evidence, hashes and attestations in selected governance surfaces, and deterministic replay

or drift checks where specified. It does not support the stronger certificate described in the earlier draft: this paper does not claim per-command cryptographic diffs, a universal signed `proof.json`, GPG/SSH signing, arbitrary credential filtering, or complete interception of an agent's PTY. Those are possible future extensions and are excluded from the present empirical claims.

6 Evaluation Protocol

The artifact defines a prospective evaluation protocol, not completed empirical evidence. Its checked-in aggregate JSON and run records are generated by a stochastic simulator and are retained as harness fixtures; they must not be cited as observations from LLM agents or human reviewers. A frozen protocol tag or commit should be recorded before empirical execution begins.

6.1 Experimental Setup

The planned suite contains 35 multi-step software-engineering tasks across seven categories. Each task is intended to run five times per condition, yielding 175 runs per condition and 525 runs overall. The final paper must record the actual model identifier, version, decoding settings, task ordering, random seeds, host environment, and exclusions at execution time; the present draft does not treat the placeholder model configuration as a measured fact.

The three conditions are:

- **Baseline A (Prompt-Only)**: the agent receives the task prompt and uses the baseline execution environment. Direct work on a protected branch is not a recommended production practice; this condition is an experimental control whose exact permissions must be documented.
- **Baseline B (Checklist-Disciplined)**: the agent receives the same task plus a persistent checklist, with no Decapod governance state.
- **Treatment (Decapod-Governed)**: the agent uses Decapod's session, todo claim, orientation, workspace, validation, proof, and completion paths. The treatment must report which optional container and proof configurations were enabled.

6.2 Metrics and Analysis

Primary outcomes are task success against an independent evaluator, invalid completion claims, recovery success after an injected interruption, and uncoordinated workspace or merge conflicts. A secondary outcome is human audit time, which requires a separately approved reviewer protocol, blinded assignment, correctness checks, and an analysis plan. All denominators and missing-run rules must be fixed before collecting results.

The current synthetic fixtures illustrate the schema only. They are not a simulation-based estimate of Decapod performance, and the paper reports no treatment effect, confidence interval, or significance claim until real runs and reviewer data are deposited. Results should be reported with per-task paired summaries, uncertainty intervals, failure categories, and overhead measurements for workspace creation, validation, proof execution, and recovery.

6.3 Falsifiable Predictions

The protocol tests whether the treatment lowers invalid completion claims, improves post-interruption recovery, reduces uncoordinated shared-workspace failures, and reduces audit effort. These are hypotheses, not guarantees: isolated workspaces can prevent direct write collisions while still producing later merge conflicts, and validation gates can reject unsupported completion without proving that a task's soft requirements are satisfied.

7 Discussion

The proposed evaluation is designed to test the core thesis that agentic software engineering is more than a language-modeling task; it is a distributed systems problem that requires formal control planes.

7.1 Systems Framing vs. Prompt Engineering

Historically, the primary method for improving coding agent performance has been prompt engineering—tuning system instructions, adding few-shot examples, or structuring chat outputs. While these methods improve the probability of a model generating correct syntax, they do not change the fundamental execution model. A model executing directly on a production workspace remains unbounded and unverified.

Enforcing explicit custody and proof shifts the focus from inputs (prompts) to execution boundaries and evidence. Decapod can require a claimed task to enter an isolated workspace and can block promotion paths when validation or proof requirements are missing. This is narrower than complete sandboxing: the current kernel does not guarantee that every incorrect change is detected, that every credential is inaccessible, or that every agent action is intercepted. The evaluation must therefore measure both safety benefits and governance overhead.

The findings of Gloaguen et al. [4] sharpen this argument. Their results show that repository context can be respected while still failing to improve task success and increasing inference cost. Instruction following is not equivalent to good governance: an agent may faithfully follow a poisoned, redundant, or over-broad context file. Decapod's constitution and context-capsule structure is motivated by this distinction. It aims to make authority, scope, and proof-relevant context explicit and queryable before inference, while preserving minimality as a design constraint. This is a testable architectural hypothesis, not a result established by the present pre-results artifact.

7.2 The Control Plane for Machine Work

In traditional software engineering, human developers follow processes: they checkout branches, write tests, request pull request reviews, and run CI/CD pipelines. However, these processes were designed for human timelines. An autonomous agent can run dozens of commands per minute, coordinate across multiple repositories, and execute complex workflows concurrently.

Decapod acts as a local control plane for machine work. It brings intent resolution, task ownership, workspace isolation, validation, proof events, and publication gates into a repository-native workflow. It does not itself coordinate every command issued by an arbitrary model runtime, so the boundary between Decapod evidence and the agent host remains part of the threat model.

7.3 Trust, Auditing, and the Limitations of Proof

A central hypothesis of this study is that structured proof files and trajectories will reduce human review time. In a transcript-only workflow, human reviews are cognitively demanding: the developer must read long chat threads, try to understand why the agent chose a specific path, and then manually check if the code works. A Decapod run, however, separates the conversational noise from the systems evidence.

Programmatic proof has strict limits. Decapod evidence can show that configured commands ran and what they returned, but it cannot verify soft requirements such as code elegance, architectural maintainability, or completeness of an unstated user goal. The goal is therefore to make objective checks and governance state easier to inspect, not to replace human review or convert a validation pass into a security guarantee.

8 Related Work

Our research intersects with four main areas of literature: Large Language Model agents in software engineering, repository-level context and memory, program verification/validation pipelines, and distributed coordination and audit frameworks.

8.1 LLM Agents in Software Engineering

Since the advent of scalable Transformers [13] and few-shot reasoning models like GPT-3 [2], code generation has moved from simple autocompletion to agentic orchestration. Frameworks such as Aider, AutoGPT, and Devin-like implementations have shown that wrapping LLMs in read-write-execute tool loops allows them to tackle complex, multi-file software modifications. Benchmarks such as SWE-bench evaluate agents on their ability to resolve real GitHub issues.

However, many contemporary coding-agent workflows operate in a transcript-only paradigm. They focus on improving the success rate of code generation rather than the safety, isolation, or auditability of the execution process. Our work addresses this gap, focusing on the control plane of the agent's environment rather than the model's internal prompt parameters.

8.2 Repository Context, Context Poisoning, and Memory

The closest contemporaneous empirical comparison is Gloaguen et al.'s study of `AGENTS.md` repository context files [4]. Across multiple agents and LLMs, that study reports no improvement in task success, more than 20% higher inference cost, broader exploration such as additional testing and file traversal, and substantial instruction-following. Its result is important for this paper's thesis: repository context is not inert documentation. It changes the agent's execution trajectory and resource consumption, so unnecessary or contradictory requirements can act as context poisoning even when the agent appears to follow them.

Our position is not that a longer constitution is automatically better. Decapod responds to the same risk by treating context as governed control-plane input: the constitution is structured into queryable nodes, `context.resolve` and context capsules scope what is supplied before inference, and plans preserve unknowns, human questions, stop conditions, and proof expectations. This design makes context selection and authority inspectable rather than placing an unbounded instruction file in the prompt. The ETH

Zürich findings therefore support the need to evaluate context quality, scope, cost, and behavioral side effects; they do not by themselves establish that Decapod's structure improves task success. Our evaluation should test whether structured, scoped context reduces harmful drift and review burden while avoiding the overhead identified by Gloaguen et al.

8.3 Program Verification and Sandboxing

Our custody and proof primitives build on a long history of program verification and systems sandboxing. Traditional verification systems focus on mathematical proof of correctness (e.g., using Coq, Isabelle, or TLA+). In contrast, our proof primitive focuses on pragmatic, machine-checkable verification (compilers, test suites, static analysis) that can be easily executed in standard software engineering workspaces.

Sandboxing technologies (e.g., Docker, WebAssembly, gVisor) are widely used to secure untrusted code execution. In our framework, we apply these concepts to agent tool execution. Decapod's custody primitive uses Git worktrees and shell sandboxing to isolate agent writes, preventing authority leakage and concurrent collisions.

8.4 Distributed Systems and Ledger Audits

The framing of agentic coding as a distributed systems execution problem is inspired by classic distributed consensus and audit architectures. Lamport's happenings-before relation [6] laid the groundwork for ordering events across independent computing nodes. MapReduce [3] and distributed databases demonstrated how to partition state and manage concurrency across large scale clusters.

Similarly, cryptographically bound ledgers (such as Git's tree hashes or blockchain transactions) provide conceptual inspiration for Decapod's Trajectory and Proof primitives. Decapod's current contribution is narrower: it journals selected governance and proof events and exposes validation, provenance, and attestation surfaces; it does not claim to log every tool call or sign one universal execution ledger.

9 Conclusion

As LLM-based coding agents transition from suggestion to delegated execution, we must treat agentic software engineering as a distributed systems execution problem rather than a simple text-generation task. The traditional transcript-only workflow—relying solely on chat transcripts and code diffs—fails to provide the safety, recovery, and verification guarantees required for enterprise codebases.

This paper formalized four primitives for accountable agentic execution:

- **Intent**: Durable task objectives and constraints.
- **Custody**: Bounded execution sandboxes and permission limits.
- **Trajectory**: Event-backed governance records rendered for reconstruction and review.
- **Proof**: Machine-checkable validation, proof-run, workunit, and provenance evidence.

We presented Decapod, a daemonless, local-first governance kernel, as a reference implementation of this framework. The implementation mapping and evaluation protocol are grounded in the current codebase, while quantitative results remain future work: the checked-in numerical fixtures are synthetic and are not evidence of treatment effects. The protocol is designed to test whether task ownership, isolated workspaces, validation, and proof gates reduce invalid completion claims, improve recovery, and reduce audit effort, subject to the stated limitations.

Future work includes executing and independently auditing the benchmark, scaling these primitives to multi-repository orchestration, developing stronger isolation layers for agent runtime processes, and exploring methods for validating complex, soft software requirements. Decapod demonstrates one repo-native design, not a complete security boundary or a universal solution.

References

[1] S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1–7):107–117, 1998.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008. First presented at OSDI 2004. ACM journal publication year is 2008.

[4] T. Gloaguen, N. Mündler, M. N. Müller, V. Raychev, and M. Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? In *MemAgents Workshop at ICLR*, 2026. Oral presentation; runner-up best paper.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.

[6] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[7] M. Minsky and S. A. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

[8] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[9] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[11] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[12] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.